NEXAFORGE AI

Model Context Protocol Integration Blueprint

A secure architecture for connecting AI applications to tools, resources, and prompts through MCP-compatible servers

Document ID	Version	Owner	Classification
NF-AI-010	2.0	AI Integration Architecture	Confidential - Internal Use

Purpose

Define how NexaForge AI integrates Model Context Protocol-compatible servers into desktop assistants, IDE agents, and backend services. The blueprint focuses on trust boundaries, server discovery, capability approval, authentication, tool execution, resource access, prompt assets, observability, and lifecycle management.

Keywords: MCP, Model Context Protocol, tools, resources, servers, security, integration

Document Control

Field	Value
Status	Approved methodology baseline
Effective date	21 July 2026
Review cycle	Every 12 months or upon material change
Primary owner	AI Integration Architecture
Related standards	Security, privacy, software delivery, incident response, and data governance standards
Supersedes	Version 1.0 concise edition

How to Use This Document

Teams should use this document as a design and review guide, not as a substitute for engineering judgment. Sections describe mandatory controls, recommended practices, decision points, and evidence expected at release. Tailoring is permitted when documented and approved by the accountable owner. Any deviation that increases risk must include compensating controls and a review date.

1. Purpose and Objectives

Define how NexaForge AI integrates Model Context Protocol-compatible servers into desktop assistants, IDE agents, and backend services. The blueprint focuses on trust boundaries, server discovery, capability approval, authentication, tool execution, resource access, prompt assets, observability, and lifecycle management.

2. Scope and Applicability

This standard applies to new implementations, major changes, vendor replacements, and material expansions of systems that rely on model context protocol integration blueprint. It covers prototypes that process real organizational data, pre-production pilots, and production services. Small experiments using synthetic data may follow a reduced process, but they must still document ownership, purpose, and data handling. The methodology does not replace legal, security, privacy, or domain-specific requirements; instead, it provides the engineering structure through which those requirements are implemented and verified.

3. Core Principles

Treat every server as a dependency: MCP servers require ownership, versioning, security review, and monitoring.

Capabilities are explicitly granted: Clients expose only approved tools and resources to a model for a given task.

The model is not the authorization layer: Authentication and authorization are enforced by the client, gateway, and target system.

Inputs and outputs are untrusted: Validate schemas, sanitize content, and protect against prompt injection from resources.

Separate read and write paths: Write actions require stronger permissions, confirmation, and audit trails.

4. Lifecycle and Detailed Procedure

1. Use-case definition

Identify which external systems must be accessed, what tasks are enabled, required data, and whether actions are read-only or mutating.

- Required evidence: a reviewable artifact or trace showing that use-case definition was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

2. Server selection or development

Review server ownership, implementation language, transport, supported capabilities, update cadence, and dependency security. Prefer narrowly scoped servers.

- Required evidence: a reviewable artifact or trace showing that server selection or development was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

3. Trust-boundary design

Map client, model, server, network, credential store, and target system. Define where user identity and tenant context are enforced.

- Required evidence: a reviewable artifact or trace showing that trust-boundary design was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

4. Capability manifest review

Inventory tools, resources, and prompts. Approve names, descriptions, schemas, permissions, side effects, and data classifications.

- Required evidence: a reviewable artifact or trace showing that capability manifest review was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Authentication and secrets

Use short-lived credentials, OAuth or workload identity where possible, encrypted secret storage, and per-environment configuration. Never place secrets in prompts.

- Required evidence: a reviewable artifact or trace showing that authentication and secrets was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

6. Client integration

Configure server discovery, connection health, timeouts, concurrency, and capability filtering. Present the model with only task-relevant capabilities.

- Required evidence: a reviewable artifact or trace showing that client integration was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

7. Invocation controls

Validate arguments, enforce allowlists, apply rate limits, require confirmation for sensitive writes, and make actions idempotent where feasible.

- Required evidence: a reviewable artifact or trace showing that invocation controls was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

8. Output handling

Schema-validate responses, scan resource content for injection patterns, label provenance, and restrict how server output can influence subsequent tool calls.

- Required evidence: a reviewable artifact or trace showing that output handling was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

9. Testing and release

Test happy paths, permission denial, malformed responses, unavailable servers, slow calls, duplicate requests, and malicious resource content.

- Required evidence: a reviewable artifact or trace showing that testing and release was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

10. Operations and decommissioning

Monitor health, latency, error types, capability use, and version drift. Revoke credentials and remove client configuration when a server is retired.

- Required evidence: a reviewable artifact or trace showing that operations and decommissioning was completed.
- Decision record: assumptions, rejected alternatives, owner, and unresolved risks.
- Exit condition: measurable criteria are met before the workflow moves forward.

5. Entry Criteria

Work may enter the methodology when a named owner, defined user outcome, initial data classification, and target environment are available. Teams should also identify downstream consumers, irreversible actions, expected traffic, and the cost of wrong answers. If these inputs are unknown, the first deliverable is a discovery brief rather than an implementation.

6. Design Review Expectations

The design review should focus on concrete decisions and evidence. Reviewers inspect architecture diagrams, state or data schemas, model and tool dependencies, evaluation plans, privacy boundaries, fallback behavior, and operating metrics. Open issues must have owners and dates. A design is not approved merely because a model can demonstrate the happy path; the team must show how the system behaves with missing inputs, conflicting evidence, unavailable services, malformed outputs, and unauthorized requests.

7. Documentation and Traceability

Each release maintains a versioned record of requirements, decisions, prompts or policies, model identifiers, data or index versions, test results, known limitations, and approvers. Traceability should allow an incident responder to determine which configuration produced a specific output. Documentation must be concise enough to maintain but detailed enough for a new engineer to reproduce critical behavior.

8. Change Management

Changes are categorized as low, medium, or high impact. Low-impact changes include wording or monitoring adjustments with no behavior change. Medium-impact changes affect routing, prompts, thresholds, or non-sensitive data. High-impact changes include model replacement, new write capabilities, new sensitive data, changed jurisdictions, or altered decision authority. Medium and high-impact changes require targeted regression testing; high-impact changes repeat the relevant risk and approval reviews.

9. Operational Readiness

Before launch, the team confirms monitoring, alert ownership, on-call procedures, rollback, rate limits, access controls, data retention, incident communication, and user support. Synthetic probes should continuously test critical paths. Runbooks should describe both technical recovery and the user-facing behavior during degradation.

10. Continuous Improvement

Production evidence is reviewed on a regular cadence. Teams analyze failures by root cause rather than by surface symptom, distinguish model errors from retrieval, tool, data, policy, or interface errors, and prioritize improvements by user harm and frequency. Confirmed defects become regression tests. Metrics are segmented by use case, language, tenant, and risk tier to avoid hiding poor performance inside global averages.

12. Roles and Responsibilities

Role	Accountability
Integration architect	Defines topology and trust boundaries.
MCP server owner	Owns implementation and capability contracts.
Client engineer	Implements discovery, filtering, and invocation.
Security engineer	Reviews authentication, authorization, and abuse cases.
Application owner	Approves business use and user experience.
SRE	Monitors availability and incidents.

13. Measurement Framework

Metric	Definition and Use
Connection success	Healthy client-server sessions.
Tool invocation success	Calls returning schema-valid expected results.
Permission denial correctness	Unauthorized requests blocked.
Write confirmation compliance	Sensitive actions executed only after required approval.
Server latency	P50 and P95 by capability.
Schema drift incidents	Breaking capability changes detected.
Unused capability ratio	Exposed capabilities never needed by the application.
Audit completeness	Actions with identity, inputs, outputs, and result status recorded.

Metrics must be segmented by use case and risk tier. Teams should define a baseline, target, alert threshold, and owner for each production metric. A metric without an action rule is considered observational rather than operational.

14. Worked Example

An IDE assistant connects to a source-control MCP server and an internal documentation server. For code review, the client exposes read-only repository search and documentation resources. Creating a pull request is hidden until the user explicitly asks for it, then requires confirmation and a scoped token. Documentation content is labeled as untrusted evidence so embedded instructions cannot trigger repository actions.

15. Risk Register and Controls

Risk	Primary Controls
Overbroad servers	Split servers by domain and capability, and expose least privilege.
Prompt injection through resources	Treat resource text as data, isolate instructions, and require policy checks before follow-on actions.
Credential leakage	Keep secrets outside model context and redact logs.
Schema drift	Pin versions, validate manifests, and run compatibility tests.
Duplicate side effects	Use idempotency keys and confirmation states.
Shadow integrations	Maintain an approved server registry and block unreviewed endpoints.

16. Release Gate

- A named business and technical owner has approved the intended use and operating boundaries.
- Required architecture, security, privacy, and risk reviews are complete.
- Evaluation results meet minimum thresholds and no severe unresolved defect remains.
- Monitoring, alerting, rollback, and incident procedures have been tested.
- User-facing disclosures, approval checkpoints, and fallback behavior are implemented where required.
- All model, prompt, tool, data, index, and policy versions are recorded in the release manifest.

17. Review Checklist

- Is the user outcome specific, measurable, and appropriate for AI assistance?
- Are authority boundaries and prohibited actions explicit?
- Can every important output be traced to inputs, evidence, and configuration?
- Does the design handle missing, conflicting, stale, or malicious inputs?
- Are sensitive data, permissions, retention, and deletion controlled?
- Are severe failures represented in the evaluation suite?
- Can the service degrade or stop safely when dependencies fail?

- Are metrics connected to owners and response procedures?

18. Appendices

MCP server review checklist

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Model Context Protocol Integration Blueprint in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Capability manifest template

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Model Context Protocol Integration Blueprint in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Threat model prompts

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Model Context Protocol Integration Blueprint in a reproducible manner.

- Owner and review date
- Inputs and dependencies
- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Operational runbook

This appendix should be maintained as a project-specific artifact. It records the concrete implementation details, decisions, evidence, and approvals needed to apply the Model Context Protocol Integration Blueprint in a reproducible manner.

- Owner and review date
- Inputs and dependencies

- Decision criteria
- Required evidence
- Known limitations and follow-up actions

Revision History

Version	Date	Summary
1.0	21 July 2026	Initial concise methodology edition.
2.0	21 July 2026	Expanded edition with detailed lifecycle, controls, roles, metrics, examples, risks, release gates, and appendices.